

MODULE 15

Recommender Systems

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Recommender Systems

What are Recommender Systems?

Recommender systems predict user preferences and suggest relevant items.

Types:

Collaborative Filtering: based on user behaviour

Content-Based: based on item features

Hybrid: combines both approaches

Knowledge-Based: domain expert rules

Applications: Netflix, Amazon, Spotify, YouTube

Collaborative Filtering

```
# User-based collaborative filtering
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Create user-item matrix
user_item = df.pivot(index='user_id', columns='item_id', values='rating').fillna(0)

# User similarity
user_sim = cosine_similarity(user_item)
user_sim_df = pd.DataFrame(user_sim, index=user_item.index, columns=user_item.index)

# Find similar users for user_id=1
similar_users = user_sim_df[1].sort_values(ascending=False)[1:6]
print('Most similar users:', similar_users)
```

Matrix Factorisation

```
from surprise import SVD, Dataset, Reader, accuracy
from surprise.model_selection import cross_validate

# Load data
reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df[['user_id', 'item_id', 'rating']], reader)

# SVD (Singular Value Decomposition)
svd = SVD(n_factors=50, n_epochs=20, lr_all=0.005, reg_all=0.02)
cross_validate(svd, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)

# Predict a rating
svd.fit(data.build_full_trainset())
prediction = svd.predict(user_id=1, iid=item_id)
print(f'Predicted rating: {prediction.est:.2f}')
```

Content-Based Filtering

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Movie recommendation by description
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(movies['description'])

# Compute cosine similarity between movies
cosine_sim = cosine_similarity(tfidf_matrix)

def recommend(title, n=10):
    idx = movies[movies['title'] == title].index[0]
    sim_scores = sorted(enumerate(cosine_sim[idx]), key=lambda x: x[1], reverse=True)
    movie_indices = [i[0] for i in sim_scores[1:n+1]]
    return movies['title'].iloc[movie_indices]
```

Evaluation Metrics

Accuracy Metrics:

RMSE: Root Mean Squared Error (rating prediction)

MAE: Mean Absolute Error

Ranking Metrics:

Precision@K: relevant items in top K recommendations

Recall@K: fraction of relevant items retrieved

NDCG: Normalised Discounted Cumulative Gain

MRR: Mean Reciprocal Rank

Map@K: Mean Average Precision

Business Metrics:

Click-through rate (CTR)

Conversion rate

Revenue lift

User engagement